ขั้นตอนการทำ dataset

all_197 · 197 โมเดล · pass 118 / fail 79 · ตอนนี้รอผลจาก Rodin

สร้างโมเดล พิสูจน์ ทำ dataset ใช้งาน

1. ภาพรวม

1. สร้าง sequence diagram — ทำที่: Mac

pass: diagram ถูก · fail: ใส่จุดผิดใน diagram (ช่วงเวลากลับด้าน หรือ mod กับตัวนับ loop)

`gen_batch.py · redo_round1.py → .drawio.xml`

▼

2. อ่าน diagram กลับมาตรวจ — ทำที่: Mac

ผู้ส่ง ผู้รับ data เวลา เงื่อนไข ต้องตรงกับที่ตั้งใจ — ไม่ตรง → ทิ้งแล้วสุ่มใหม่ (กลับไปข้อ 1)

`check_parse() ใน gen_batch.py`

▼

3. แปลงด้วย ChronoSeq-B — ทำที่: Mac

ใช้สูตรแปลงตามปกติ ไม่แก้ Event-B ภายหลัง แล้วคำนวณ PO ที่คาดว่าจะล้ม

`→ _EventB.txt · _Rodin.zip · meta.json`

▼

4. จัด folder — ทำที่: Mac

แยกตามหัวข้อ และ pass / fail · รวม Rodin project เป็น zip เดียว · ทำ PDF อธิบาย

`organize_batches.py → batches/all_197/`

▼

— ส่ง Rodin_Import_all_197.zip ไปเครื่อง Windows —

5. พิสูจน์ด้วย Rodin — ตอนนี้อยู่ตรงนี้ — ทำที่: Windows

import zip → auto-prover (Atelier B ML/PP) รันจนครบ → zip workspace ส่งกลับ

`Rodin 3.9 + Atelier B`

▼

— ส่ง workspace ที่พิสูจน์แล้วกลับมา —

6. รวมผลเข้า dataset — ทำที่: Mac

อ่านผลจาก .bps ของแต่ละ PO · รอบแรกใช้ ID เดิม ชุดใหม่ได้ model_049–198

`integrate_batch.py → models/ · sources/ · rodin_raw/`

▼

7. เทียบผล Rodin กับที่ออกแบบ — ทำที่: Mac

ตรงกัน → ใช้ได้ · ไม่ตรง → ดูว่า PO ไหนต่าง แล้วตรวจ diagram / สูตรแปลง

`rebuild_labels.py`

▼

8. ได้ dataset — ทำที่: Mac

label ของทุกโมเดล (total / discharged / failed + ราย PO) · แยก folder ตามผลจริง · ตารางหัวข้อ

`dataset.jsonl · by_result/ · model_categories.csv`

▼

9. ใช้กับ LLM — ทำที่: —

input = model.txt (+ features) · output = จำนวน PO ที่พิสูจน์ได้ · เทียบกับ label

`ห้ามใช้ meta.json · sources/ · rodin_raw/ เป็น input`

2. โมเดล fail เกิดขึ้นอย่างไร

จุดผิดอยู่ใน **diagram** เท่านั้น สูตรแปลงทำงานตามปกติ แล้ว PO ข้อที่เกี่ยวข้องจะพิสูจน์ไม่ได้

ใน diagram	สูตรแปลงได้	Rodin
`notify() {5..3}` ช่วงเวลากลับด้าน (5 > 3)	`t_time_i :∈ 5..3` ไม่มีค่าให้เลือก	**FIS ล้ม** `INITIALISATION/actN/FIS`
ใน diagram	สูตรแปลงได้	Rodin
loop `[retryNo < 5]` + alt/opt `[retryNo mod 2 == 0]`	guard `retryNo mod 2 = 0` แต่ invariant มีแค่ `retryNo ∈ ℤ`	**WD ล้ม** (ต้องรู้ว่า retryNo ≥ 0) `sendX/grdN/WD`

INV ไม่ล้มจาก diagram เพราะสูตรแปลงสร้างแค่ invariant บอกชนิดข้อมูลกับ `inv5: receivedMessages ⊆ sentMessages` และใส่ guard รองรับ inv5 ให้เองเสมอ

3. แต่ละขั้นใช้อะไร ได้อะไร

ขั้น	ทำที่	เครื่องมือ / ไฟล์ที่ได้
1. สร้าง sequence diagram	Mac	`gen_batch.py · redo_round1.py → .drawio.xml`
2. อ่าน diagram กลับมาตรวจ	Mac	`check_parse() ใน gen_batch.py`
3. แปลงด้วย ChronoSeq-B	Mac	`→ _EventB.txt · _Rodin.zip · meta.json`
4. จัด folder	Mac	`organize_batches.py → batches/all_197/`
5. พิสูจน์ด้วย Rodin	Windows	`Rodin 3.9 + Atelier B`
6. รวมผลเข้า dataset	Mac	`integrate_batch.py → models/ · sources/ · rodin_raw/`
7. เทียบผล Rodin กับที่ออกแบบ	Mac	`rebuild_labels.py`
8. ได้ dataset	Mac	`dataset.jsonl · by_result/ · model_categories.csv`
9. ใช้กับ LLM	—	`ห้ามใช้ meta.json · sources/ · rodin_raw/ เป็น input`

script ทั้งหมดอยู่ใน `EventB_PO_Dataset/tools/` (ตัวสร้างโมเดลอยู่ใน `tools/generators/`)